Output Screenshots

Ensemble Learning for Text Classification

Notebook: Ensemble_Text_Classification.ipynb

Captured outputs of every executed code cell

In [1]: ran in 1.3s

```
import time, warnings, re
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.decomposition import TruncatedSVD
from sklearn.pipeline import make_pipeline
from sklearn.metrics import accuracy_score, f1_score, confusion_matrix, classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.naive_bayes import MultinomialNB
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (RandomForestClassifier, BaggingClassifier, AdaBoostClassifier,
                              HistGradientBoostingClassifier, VotingClassifier, StackingClassifier)

warnings.filterwarnings("ignore")
SEED = 42
np.random.seed(SEED)
... (2 more lines)
```

Out [1]:

```
Setup complete.
```

In [2]: ran in 6.0s

```
CLASSES = ["sports", "technology", "health", "finance", "politics"]

def make_synthetic_corpus(n_per_class=600, seed=SEED):
    rng = np.random.default_rng(seed)
    topic = {
      "sports": "game team season player coach league score win match goal training stadium champion fans tournament defense inju
      "technology": "software data network system algorithm device cloud server chip code users platform security update model in
      "health": "patient doctor hospital treatment disease symptoms medicine clinical diet risk therapy blood study care health s
      "finance": "market stock bank investors profit economy shares rate fund trade growth price inflation budget earnings credit
      "politics": "government election vote policy senate president law congress party campaign minister bill court tax debate re
    }
    common = ("the of and to in is that for it with as was on be at by this have from or one had not but what all were when we th
              "their said if do will each about how up out them then she many some so these would other into has more her two lik
              "than first been its who now people my made over did down only way find use may water long little very after words
    syll = ["ka","lo","mi","ra","ten","vo","sa","nu","pe","dor","qui","bel","zan","tor","mar","fi","gro","lin","pa","shu"]
    rare = ["".join(rng.choice(syll, size=rng.integers(2, 4))) for _ in range(1800)]
    rare = list(dict.fromkeys(rare))
    R = len(rare)
    aff = {}
    for c in CLASSES:
        w = rng.permutation(R).astype(float)
        w = 1.0 / (1.0 + w) ** 0.9
... (40 more lines)
```

Out [2]:

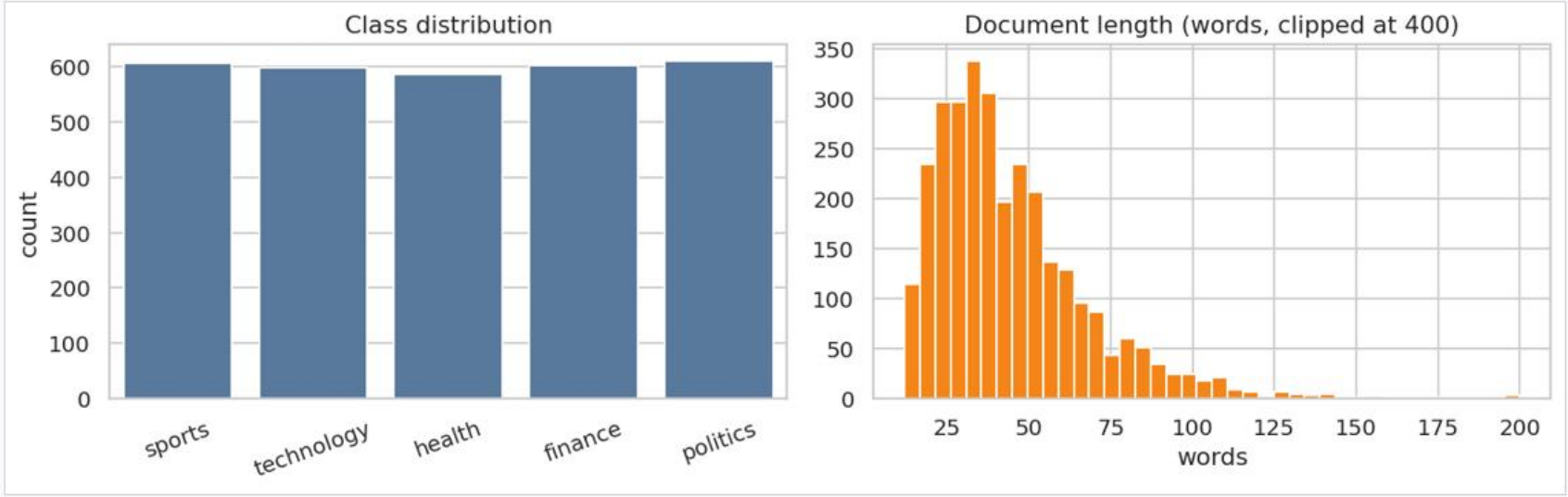
```
Dataset used : Synthetic 5-topic corpus (offline fallback)
Documents    : 3000 | Classes: ['sports', 'technology', 'health', 'finance', 'politics']
Example doc  : it quiquidor senate little there torgro shumarka fiquipa midortor market kaminu investors tordorlo congress which
n losami lofi nuvoka quishushu gropeten  ...
```

In [3]: ran in 0.3s

```
lengths = np.array([len(t.split()) for t in texts])
fig, ax = plt.subplots(1, 2, figsize=(11, 3.6))
sns.countplot(x=[CLASSES[i] for i in y], order=CLASSES, ax=ax[0], color="#4C78A8")
ax[0].set_title("Class distribution"); ax[0].set_xlabel(""); ax[0].tick_params(axis="x", rotation=20)
ax[1].hist(np.clip(lengths, 0, 400), bins=40, color="#F58518")
ax[1].set_title("Document length (words, clipped at 400)"); ax[1].set_xlabel("words")
plt.tight_layout(); plt.show()
print(f"Median length: {np.median(lengths):.0f} words | Mean: {lengths.mean():.0f}")
```

Out [3]:

```
Median length: 39 words | Mean: 45
```



In [4]: ran in 0.7s

```
X_tr_txt, X_te_txt, y_tr, y_te = train_test_split(texts, y, test_size=0.25, stratify=y, random_state=SEED)

tfidf = TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_df=0.9, sublinear_tf=True)
Xtr = tfidf.fit_transform(X_tr_txt)
Xte = tfidf.transform(X_te_txt)

svd = TruncatedSVD(n_components=100, random_state=SEED)
Ztr = svd.fit_transform(Xtr)
Zte = svd.transform(Xte)

print("Train / test docs :", Xtr.shape[0], "/", Xte.shape[0])
print("TF-IDF features   :", Xtr.shape[1], f"(sparsity {100*(1-Xtr.nnz/np.prod(Xtr.shape)):.1f}%)")
print("SVD explained var :", f"{svd.explained_variance_ratio_.sum():.2%} with 100 components")
```

Out [4]:

```
Train / test docs : 2250 / 750
TF-IDF features   : 12357 (sparsity 99.6%)
SVD explained var : 15.43% with 100 components
```

In [6]: ran in 0.6s

```
lr  = evaluate("Logistic Regression", "Baseline", LogisticRegression(C=10, max_iter=2000), Xtr, Xte)
nb  = evaluate("Multinomial NB", "Baseline", MultinomialNB(alpha=0.1), Xtr, Xte)
svm = evaluate("Linear SVM", "Baseline", LinearSVC(C=1.0), Xtr, Xte)
dt  = evaluate("Decision Tree (single)", "Baseline", DecisionTreeClassifier(max_depth=None, random_state=SEED), Ztr, Zte)
```

Out [6]:

```
Logistic Regression          acc=0.8747  macro-F1=0.8748  fit=  0.21s
Multinomial NB               acc=0.7653  macro-F1=0.7650  fit=  0.00s
Linear SVM                   acc=0.8773  macro-F1=0.8775  fit=  0.03s
Decision Tree (single)       acc=0.8387  macro-F1=0.8389  fit=  0.36s
```

In [7]: ran in 30.7s

```
rf = evaluate("Random Forest (TF-IDF)", "Bagging",
              RandomForestClassifier(n_estimators=200, max_features="sqrt", n_jobs=-1, random_state=SEED), Xtr, Xte)
rf_svd = evaluate("Random Forest (SVD)", "Bagging",
              RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=SEED), Ztr, Zte)
bag_lr = evaluate("Bagged Logistic Reg.", "Bagging",
              BaggingClassifier(LogisticRegression(C=10, max_iter=1000), n_estimators=15, random_state=SEED), Xtr, Xte)
bag_dt = evaluate("Bagged Decision Trees", "Bagging",
              BaggingClassifier(DecisionTreeClassifier(), n_estimators=100, random_state=SEED), Ztr, Zte)
```

Out [7]:

```
Random Forest (TF-IDF)      acc=0.8880  macro-F1=0.8881  fit=  4.59s
Random Forest (SVD)         acc=0.8853  macro-F1=0.8854  fit=  3.65s
Bagged Logistic Reg.        acc=0.8760  macro-F1=0.8761  fit=  3.75s
Bagged Decision Trees       acc=0.8787  macro-F1=0.8788  fit= 18.58s
```

In [8]: ran in 13.6s

```
ada = evaluate("AdaBoost (stumps, SVD)", "Boosting",
               AdaBoostClassifier(DecisionTreeClassifier(max_depth=2), n_estimators=200, learning_rate=0.5,
                                  random_state=SEED), Ztr, Zte)
hgb = evaluate("Hist. Gradient Boosting (SVD)", "Boosting",
               HistGradientBoostingClassifier(max_iter=150, learning_rate=0.1, max_depth=4, random_state=SEED), Ztr, Zte)
```

Out [8]:

```
AdaBoost (stumps, SVD)       acc=0.8813  macro-F1=0.8814  fit= 10.92s
Hist. Gradient Boosting (SVD) acc=0.8893  macro-F1=0.8894  fit=  2.59s
```

In [9]: ran in 2.5s

```
def base_models():
    return [("lr", LogisticRegression(C=10, max_iter=2000)),
            ("nb", MultinomialNB(alpha=0.1)),
            ("svm", CalibratedClassifierCV(LinearSVC(C=1.0), cv=3))]

hard = evaluate("Voting (hard)", "Voting", VotingClassifier(base_models(), voting="hard"), Xtr, Xte)
soft = evaluate("Voting (soft)", "Voting", VotingClassifier(base_models(), voting="soft"), Xtr, Xte)
stack = evaluate("Stacking (LR meta)", "Voting",
                 StackingClassifier(base_models(), final_estimator=LogisticRegression(max_iter=1000),
                                    cv=StratifiedKFold(5, shuffle=True, random_state=SEED)), Xtr, Xte)
```

Out [9]:

```
Voting (hard)               acc=0.8760  macro-F1=0.8761  fit=  0.31s
Voting (soft)               acc=0.8760  macro-F1=0.8763  fit=  0.29s
Stacking (LR meta)          acc=0.8760  macro-F1=0.8762  fit=  1.88s
```

In [10]: ran in 0.0s

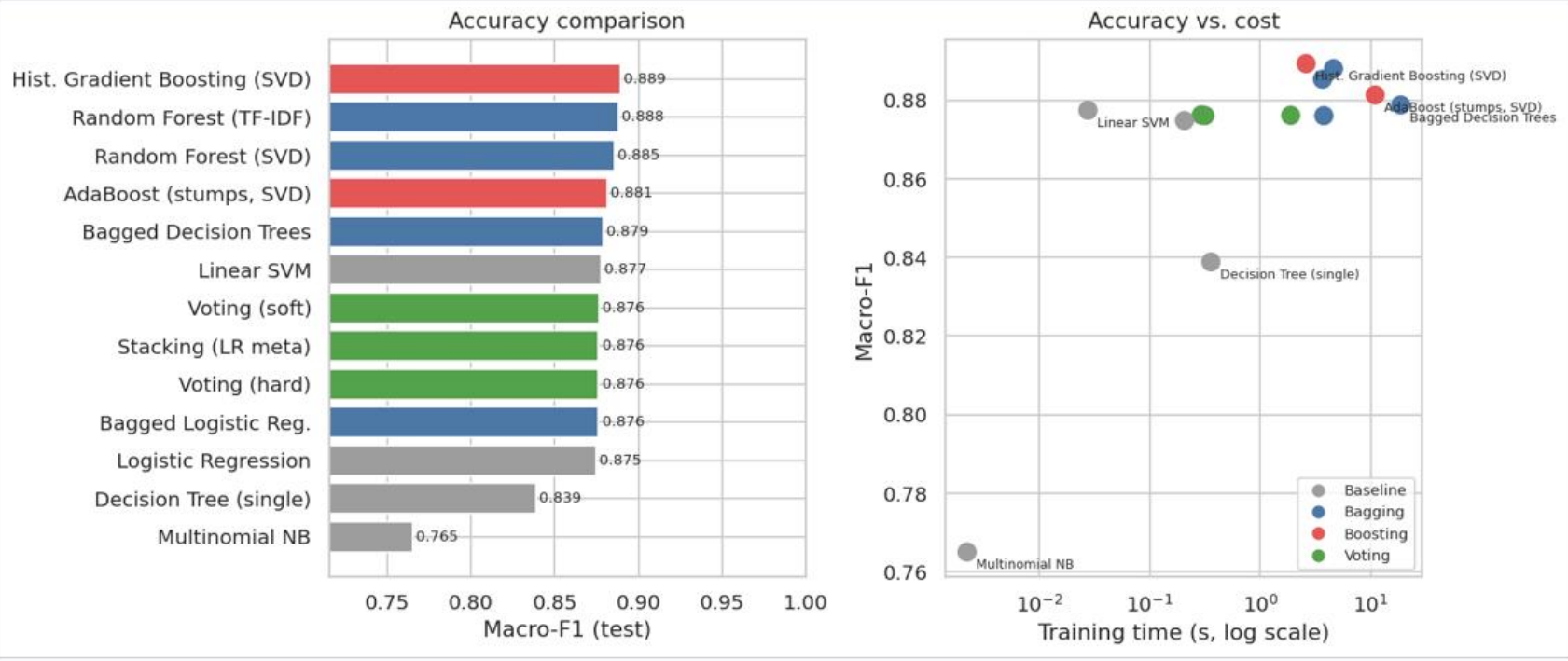
```
res = pd.DataFrame(RESULTS).sort_values("MacroF1", ascending=False).reset_index(drop=True)
res.to_csv("results.csv", index=False)
print(res.round(4).to_string())
```

Out [10]:

	Model	Family	Accuracy	MacroF1	Fit_s	Predict_s
0	Hist. Gradient Boosting (SVD)	Boosting	0.8893	0.8894	2.5915	0.0330
1	Random Forest (TF-IDF)	Bagging	0.8880	0.8881	4.5874	0.0631
2	Random Forest (SVD)	Bagging	0.8853	0.8854	3.6475	0.0307
3	AdaBoost (stumps, SVD)	Boosting	0.8813	0.8814	10.9187	0.0387
4	Bagged Decision Trees	Bagging	0.8787	0.8788	18.5825	0.0260
5	Linear SVM	Baseline	0.8773	0.8775	0.0275	0.0004
6	Voting (soft)	Voting	0.8760	0.8763	0.2921	0.0041
7	Stacking (LR meta)	Voting	0.8760	0.8762	1.8794	0.0052
8	Voting (hard)	Voting	0.8760	0.8761	0.3143	0.0065
9	Bagged Logistic Reg.	Bagging	0.8760	0.8761	3.7504	0.0128
10	Logistic Regression	Baseline	0.8747	0.8748	0.2056	0.0006
11	Decision Tree (single)	Baseline	0.8387	0.8389	0.3566	0.0005
12	Multinomial NB	Baseline	0.7653	0.7650	0.0022	0.0005

In [11]: ran in 0.5s

```
colors = {"Baseline": "#9D9D9D", "Bagging": "#4C78A8", "Boosting": "#E45756", "Voting": "#54A24B"}
order = res.sort_values("MacroF1")
fig, ax = plt.subplots(1, 2, figsize=(12, 5.2))
ax[0].barh(order.Model, order.MacroF1, color=[colors[f] for f in order.Family])
ax[0].set_xlim(max(0, order.MacroF1.min() - 0.05), 1.0); ax[0].set_xlabel("Macro-F1 (test)")
ax[0].set_title("Accuracy comparison")
for i, v in enumerate(order.MacroF1): ax[0].text(v + 0.002, i, f"{v:.3f}", va="center", fontsize=8)
ax[1].scatter(res.Fit_s, res.MacroF1, c=[colors[f] for f in res.Family], s=90)
label_these = ["Multinomial NB", "Decision Tree (single)", "Linear SVM", "Bagged Decision Trees", "Hist. Gradient Boosting (SVD)"
for _, r in res.iterrows():
    if r.Model in label_these: ax[1].annotate(r.Model, (r.Fit_s, r.MacroF1), fontsize=7, xytext=(5, -9), textcoords="offset point
ax[1].set_xscale("log"); ax[1].set_xlabel("Training time (s, log scale)"); ax[1].set_ylabel("Macro-F1")
ax[1].set_title("Accuracy vs. cost")
handles = [plt.Line2D([0], [0], marker="o", ls="", color=c, label=k) for k, c in colors.items()]
ax[1].legend(handles=handles, loc="lower right", fontsize=8)
plt.tight_layout(); plt.show()
```



In [12]: ran in 0.1s

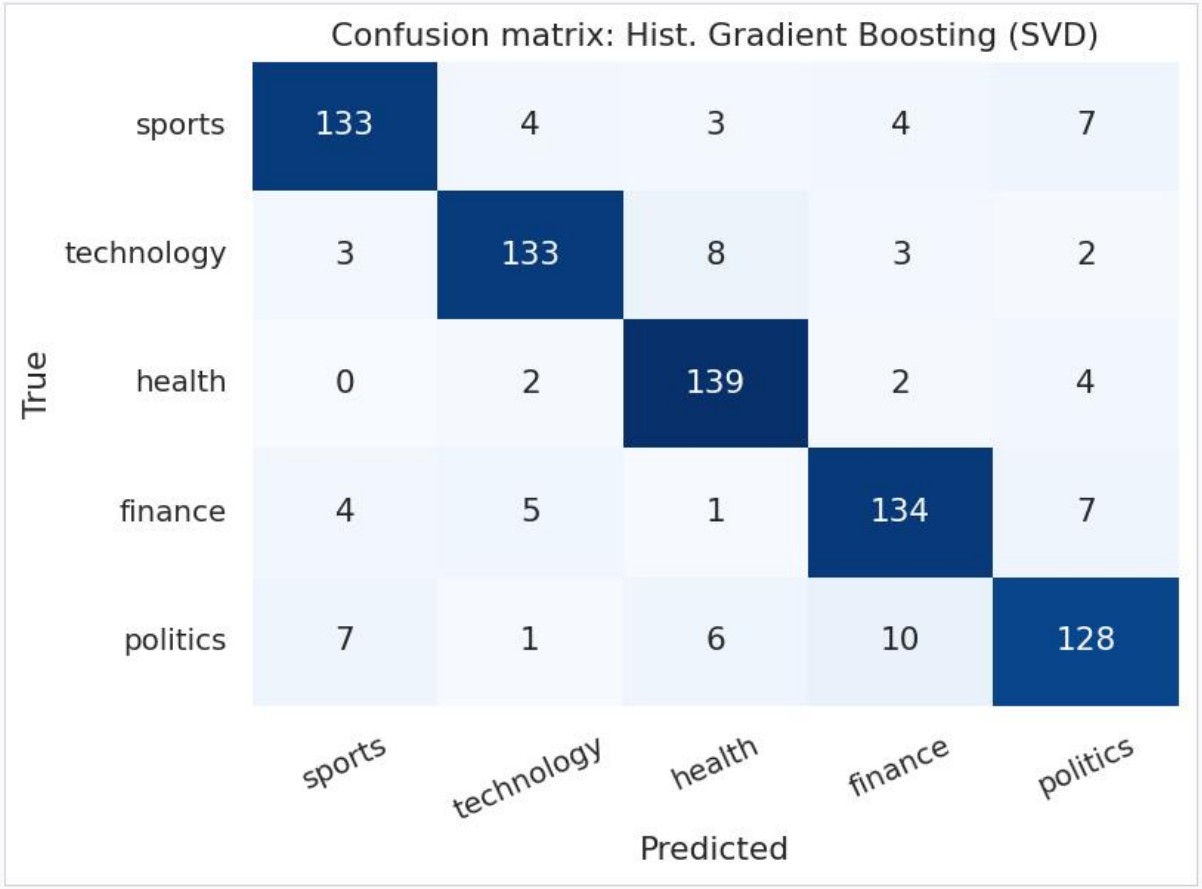
```
best = res.iloc[0].Model
print("Best model:", best)
print(classification_report(y_te, PREDS[best], target_names=CLASSES, digits=3))
cm = confusion_matrix(y_te, PREDS[best])
fig, ax = plt.subplots(figsize=(6.4, 4.8))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=CLASSES, yticklabels=CLASSES, ax=ax, cbar=False)
ax.set_xlabel("Predicted"); ax.set_ylabel("True"); ax.set_title(f"Confusion matrix: {best}")
plt.xticks(rotation=25); plt.yticks(rotation=0); plt.tight_layout(); plt.show()
```

Out [12]:

```
Best model: Hist. Gradient Boosting (SVD)
              precision    recall  f1-score   support

      sports      0.905     0.881     0.893       151
  technology      0.917     0.893     0.905       149
      health      0.885     0.946     0.914       147
     finance      0.876     0.887     0.882       151
    politics      0.865     0.842     0.853       152

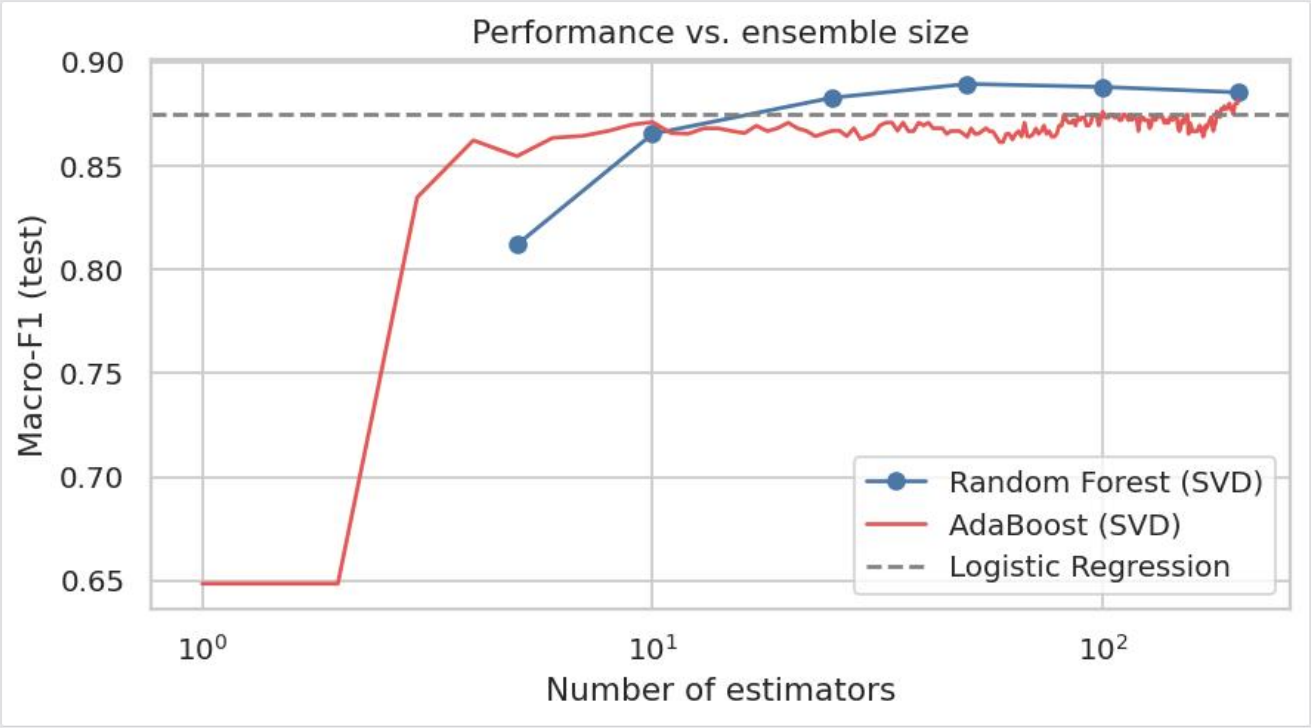
    accuracy                          0.889       750
   macro avg      0.890     0.890     0.889       750
weighted avg      0.890     0.889     0.889       750
```



In [13]: ran in 18.8s

```
sizes = [5, 10, 25, 50, 100, 200]
rf_curve = []
for n in sizes:
    m = RandomForestClassifier(n_estimators=n, n_jobs=-1, random_state=SEED).fit(Ztr, y_tr)
    rf_curve.append(f1_score(y_te, m.predict(Zte), average="macro"))
ada_big = AdaBoostClassifier(DecisionTreeClassifier(max_depth=2), n_estimators=200, learning_rate=0.5,
                             random_state=SEED).fit(Ztr, y_tr)
ada_curve = [f1_score(y_te, p, average="macro") for p in ada_big.staged_predict(Zte)]

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(sizes, rf_curve, "o-", label="Random Forest (SVD)", color="#4C78A8")
ax.plot(range(1, len(ada_curve) + 1), ada_curve, "-", label="AdaBoost (SVD)", color="#E45756")
ax.axhline(res.loc[res.Model == "Logistic Regression", "MacroF1"].iloc[0], ls="--", color="gray", label="Logistic Regression")
ax.set_xscale("log"); ax.set_xlabel("Number of estimators"); ax.set_ylabel("Macro-F1 (test)")
ax.set_title("Performance vs. ensemble size"); ax.legend(); plt.tight_layout(); plt.show()
```



In [14]: ran in 12.0s

```
rng = np.random.default_rng(SEED)
def boot_diff(a, b, n=1000):
    diffs = []
    for _ in range(n):
        idx = rng.integers(0, len(y_te), len(y_te))
        diffs.append(f1_score(y_te[idx], PREDS[a][idx], average="macro") - f1_score(y_te[idx], PREDS[b][idx], average="macro"))
    diffs = np.array(diffs)
    return diffs.mean(), np.percentile(diffs, 2.5), np.percentile(diffs, 97.5)

for a, b in [("Voting (soft)", "Logistic Regression"), ("Stacking (LR meta)", "Logistic Regression"),
             ("Random Forest (TF-IDF)", "Logistic Regression"), ("Hist. Gradient Boosting (SVD)", "Logistic Regression")]:
    m, lo, hi = boot_diff(a, b)
    verdict = "significant" if (lo > 0 or hi < 0) else "not significant"
    print(f"{a:<32s} minus {b:<20s}: {m:+.4f}  95% CI [{lo:+.4f}, {hi:+.4f}]  -> {verdict}")
```

Out [14]:

```
Voting (soft)                    minus Logistic Regression : +0.0015  95% CI [-0.0079, +0.0105]  -> not significant
Stacking (LR meta)               minus Logistic Regression : +0.0015  95% CI [-0.0056, +0.0084]  -> not significant
Random Forest (TF-IDF)           minus Logistic Regression : +0.0132  95% CI [-0.0036, +0.0300]  -> not significant
Hist. Gradient Boosting (SVD)    minus Logistic Regression : +0.0152  95% CI [-0.0044, +0.0325]  -> not significant
```